

cuTile 기반 고성능 LLM 추론 엔진 최적화 및 코딩 에이전트 협력 설계 실증 연구

*Optimizing High-Performance LLM Inference Engine via cuTile and Human-Agent
Co-Design: An Empirical Study*

병렬 및 분산 시스템 연구실

한국정보과학회 학술대회 투고 드래프트

요약 (ABSTRACT)

거대 언어 모델(LLM) 서빙 엔진의 지연 시간 및 처리량 성능은 GPU 메모리와 연산 대역폭 병목을 해결하기 위한 하드웨어 수준의 커널 최적화에 직결된다. 그러나 vLLM 등 대다수의 고성능 서빙 프레임워크는 Linux 환경에 강력하게 종속되어 있어, 연구 및 교육 목적과 로컬 온프레미스 장비가 집중된 Windows OS 생태계로의 이식성에 심각한 장벽이 존재한다. 본 연구는 Windows OS 환경에서 NVIDIA의 최신 JIT 컴파일 기반 Python DSL인 cuTile 및 CUDA Python 1.0을 도입하여 크로스 플랫폼 호환성을 보장하는 고성능 추론 엔진인 **micro-vllm**을 설계하고 구현하였다.

특히, 복잡한 GPU 커널 코딩과 E2E 시스템 설계 병목의 탐색 공간을 효율적으로 최적화하기 위해, 별도의 자율형 에이전트 루프 시스템 구축 대신 **AI 코딩 에이전트(Coding Agent)**를 공동 설계(Co-design) 및 병목 분석 파트너로 활용하는 인간-에이전트 협력적 최적화 파이프라인을 구축하였다. 본 논문은 이 파이프라인을 바탕으로 미시적 커널 수준에서 레지스터 스푼링(Register Spilling)을 유발하는 물리적 하드웨어 임계치를 규명하고 64x64 타일링 스케줄링을 도출하였으며, Paged KV 캐시 상의 dynamic shape mismatch를 sub-millisecond 수준에서 해소하는 벡터화된 논리-물리 블록 재배치 기법을 고안하였다.

나아가 시스템 이식 과정에서 JIT 컴파일 방지를 위한 dynamic padding 기법이 오히려 PyTorch caching allocator와 가비지 컬렉터의 메모리 할당 스래싱을 촉발하여 전체 생성 처리량을 67.0% 저하시키는 시스템적 부작용을 규명하고 무패딩 Eager Serving 구조를 확립하였다. 최종적으로 CUDA Green Contexts 기술을 적용하여 SM 자원을 Prefill(32 SM)과 Decode(16 SM) 영역으로 물리 격리함으로써 단일 소비자용 GPU(RTX 5070) 환경에서 **중간 값 Decode 지연시간(P50 ITL) 14.0% 단축 및 전체 토큰 처리량 10.6% 향상(459.73 tok/s)**을 달성하였다.

1. 서론 (Introduction)

최근 생성형 AI 기술의 급격한 성장에 힘입어 로컬 온프레미스 장비나 오프라인 연구실 환경에서 LLM을 구동하려는 수요가 폭증하고 있다. 특히 RAG(Retrieval-Augmented Generation), 로컬 지식베이스 구축, 그리고 기업 정보 보안을 유지하기 위한 로컬 프라이빗 추론 환경 구축은 현업 시스템 설계에 있어 필수적인 과제이다. 그러나 vLLM이나 TensorRT-LLM 등 실용적으로 널리 쓰이는 대규모 서버 아키텍처들은 Triton 커널 컴파일러, FlashAttention 패키지, 그리고 분산 텐서 통신용 NCCL 라이브러리 등 강력한 Linux 독점 환경에 강하게 묶여 있다. 이로 인해 데스크톱 컴퓨터와 로컬 서버의 보편적 운영체제인 Windows 환경에서는 WSL2(Windows Subsystem for Linux) 우회를 통한 가상화 비용과 물리 기기 다이렉트 제어 제약 등으로 최상의 연산 성능을 보장하기 어려운 한계가 존재했다.

본 연구는 이러한 크로스 플랫폼 장벽을 해소하고 최적의 로컬 LLM 서버 환경을 제공하기 위해, Windows OS 환경에서 동작하는 NVIDIA의 새로운 **cuTile** Python DSL 및 **CUDA Python 1.0** 기반의 고성능 서버 엔진인 **micro-vllm**을 개발하였다. 이 과정에서 C++ 수준의 직접적인 CUDA 프로그래밍 오버헤드와 하드웨어 프로파일링 분석의 긴 탐색 주기를 단축시키기 위해, **AI 코딩 에이전트 (Coding Agent)**와의 협력적 튜닝 파이프라인을 가동하여 시스템을 설계, 실험 및 개선해 나가는 실험과학적 엔지니어링 접근을 도입하였다.

본 논문은 단일 연산 커널 수준(Micro-level)에서의 최적화 구조가 실제 엔드투엔드(E2E) 서비스 아키텍처로 통합될 때 야기되는 '추상화의 누수(Abstraction Leak)' 병목 현상을 실증적으로 추적하고 그 대안을 제시한다. 구체적으로는 dynamic padding 최적화가 유발하는 PyTorch CUDA caching allocator의 호스트-디바이스 메모리 스래싱 문제를 규명하였으며, CUDA Green Contexts 기술을 통한 하드웨어 자원 격리 벤치마크 결과를 분석하여 디코딩의 메모리 대역폭 향상과 캐시 효율성을 입증하였다.

1.1. 본 연구의 주요 기여 (Key Contributions)

본 논문이 제시하는 학술적 및 시스템적 주요 기여점은 그 중요도 순에 따라 다음과 같다.

- **[Core Discovery] 역직관적 시스템 병목 규명 (Allocator Thrashing)**

"We uncover a counter-intuitive pitfall in eager-mode LLM serving: while shape padding is conventionally employed to amortize JIT compilation overhead, we empirically demonstrate that it induces catastrophic contention in the PyTorch CUDA caching allocator. This host-side

memory thrashing collapses end-to-end throughput by 67.0%, proving that memory allocation symmetry, not computational shape symmetry, is the primary bottleneck in Python-native serving."

JIT 방지를 위해 형태를 패딩하는 기존 상식과 직관을 뒤집고, Eager-mode LLM serving 환경 하에서 dynamic padding이 오히려 메모리 할당자의 catastrophic contention(이중 연결 리스트 스캔, cudaMalloc/cudaFree, GC 동기화 오버헤드 등)을 촉발하여 전체 처리량을 67.0%나 붕괴시킨다는 역설적인 아키텍처적 병목을 규명하였습니다.

- **[AI4Sys] 지식-주도형 코딩 에이전트 협력 프레임워크 (Human-AI Co-design Framework)**

"We introduce a human-AI co-design framework (Architect-TechLead-Executor) where a coding agent translates hardware telemetry (e.g., register spilling) into persistent DSL constraints. This enabled the efficient navigation of the complex design space, proving that structured agent collaboration accelerates low-level kernel optimization."

하드웨어 프로파일링의 물리적 한계점(예: register spilling)을 영구적인 DSL 제약 규칙으로 동적 피드백하여 컴파일 성능을 보증하는 3계층(Architect-TechLead-Executor) 협업 최적화 방법론을 제시하였습니다. 이를 통해 복잡한 GPU 커널 개발이 자율 진화 순환형(Evolutionary Closed Loop) 과학실험 프레임워크로 통제되어 가속될 수 있음을 증명하였습니다.

- **[System & Methodology] Windows-네이티브 구현 및 자원 격리 솔루션 (Windows-Native serving with Green Contexts)**

"To mitigate this, we present the first Windows-native LLM inference engine built with NVIDIA cuTile and CUDA Python 1.0. By partitioning SMs via Green Contexts (32 for prefill, 16 for decode), we isolate L2 cache residency, achieving 14.0% lower median decode latency and 10.6% higher throughput. This validates that physical resource partitioning effectively counters allocator-induced jitter."

리눅스 의존성이 심각한 서빙 환경을 극복하고 Windows 환경에서 이식 및 가속을 보증하기 위해, cuTile 및 CUDA Python 1.0 기반 고성능 엔진인 **micro-vllm**을 최초 구현했습니다. 특히 CUDA Green Contexts 기술로 SM 자원을 Prefill(32 SM)과 Decode(16 SM)로 강제 구획 분할하여 L2 캐시 residency를 격리함으로써, Decode P50 ITL을 14.0% 단축하고 Throughput을 10.6% 개선하는 물리 격리 솔루션을 정량 입증했습니다.

- **[Educational & Reproducibility] 투명한 단계별 커널 커리큘럼 (Progressive serving stack without black-box libraries)**

"Unlike production frameworks that rely on black-box libraries (FlashAttention), we construct the entire inference stack progressively—from tiled GEMM, FMHA, to PagedAttention—without external dependencies. This curriculum bridges the gap between GPU architecture education and production-grade serving, enabling full reproducibility in academic environments."

기존의 거대 서빙 라이브러리들이 의존하는 블랙박스 커널(FlashAttention 등) 종속성에서 탈피하여, 단일 GEMM부터 FMHA, Paged KV 캐시 레이어에 이르기까지 전체 추론 스택의 핵심 요소를 cuTile로 progressive하게 구현하였습니다. 이를 통해 GPU 아키텍처 교육과 실제 운영 등급의 시스템 설계 간의 지식 격차를 메우고 학계 및 교육 환경에서의 완전한 재현성(Reproducibility)을 제공합니다.

2. 관련 연구 및 배경 (Related Work & Background)

2.1. LLM 서버 엔진과 PagedAttention

LLM의 자기회귀적(Autoregressive) 생성 루프는 매 스텝마다 과거의 토큰 정보(Key, Value)를 저장하고 재사용하는 KV 캐싱을 핵심 구조로 가진다. vLLM[1]은 이 KV 캐시를 고정 크기의 물리적 페이지 블록으로 분할 관리하는 PagedAttention 기법을 통해 GPU 메모리의 외부/내부 단편화를 원천 차단하고 처리량을 혁신적으로 개선하였다. 그러나 기존 시스템들은 분산 환경을 전제로 NCCL 통신 레이어에 의존하고 있어, 단일 혹은 소비자용 GPU 환경 및 Windows와 같은 이기종 OS에서의 호환성이 현저히 떨어진다.

2.2. CUDA Python 1.0 및 cuTile DSL

최근 공식 릴리즈된 CUDA Python 1.0[2]은 Python 런타임 환경에서 하드웨어 드라이버 및 CUDA API를 직접 조작할 수 있는 안정적인 바인딩 계층을 마련하였다. 이 중 **cuTile** DSL은 행렬 곱셈이나 어텐션 연산 등의 집약 연산을 타일(Tile) 단위로 기하학적으로 추상화하여, 레지스터 및 공유 메모리 캐싱 레이아웃을 Python 코딩 수준에서 제어할 수 있도록 돕는다. 특히 CUDA 13.3부터는 Compute Capability 9.0 (Hopper) 아키텍처까지 균일하게 지원하므로, 로컬 데스크톱(RTX 40/50 시리즈)에서 작성된 어텐션 타일 커널이 클라우드 서버(H100/H200)로 추가적인 수정이나 재작성 공정 없이 투명하게 컴파일 및 포팅되는 강력한 하드웨어 호환성(Hardware Portability)을 확보할 수 있다.

2.3. 시스템 최적화 파트너로서의 코딩 에이전트

기존의 LLM 코드 생성 및 자동 최적화 연구들은 정적으로 파라미터를 탐색하는 단순 그리드 서치(Grid Search)나 선택스 에러를 수정하는 자율 루프 시스템 모델에 치중해 왔다. 그러나 하드웨어 수준의 커널 최적화는 컴파일러 메타데이터 해석, 레지스터 스펠링 감지, 호스트-디바이스 메모리 할당 특성 등 다차원적 지식의 융합이 필요하다. 본 연구에서는 자율 구동되는 루프 시스템을 직접 개발하는 대신, 개발자와 긴밀히 상호작용하는 **코딩 에이전트(Coding Agent)**를 설계 및 미시적/거시적 최적화의 피드백 파트너로 활용함으로써 복잡한 GPU 시스템 튜닝을 인간의 가이드 하에 가속하는 '컴파운드 시스템 엔지니어링(Compound System Engineering)' 방법론을 구현하였다.

3. 인지 컴파일러 및 과학실험 기반 3계층 공동 설계 방법론 (Three-Tier Co-design Methodology based on Cognitive Compiler and Controlled Scientific Experiment)

단일 GPU의 극단적인 하드웨어 자원 제약 하에서 LLM 추론 엔진 커널을 효과적으로 마이그레이션 및 최적화하기 위해서는, 고수준 아키텍처 모델과 저수준 물리 연산 간의 긴밀한 상호작용이 필수적이다. 본 연구는 시스템 개발 과정의 과학적 엄밀성과 탐색 효율성을 보장하기 위해, 커널 엔지니어링을 통제된 과학실험(Controlled Scientific Experiment)으로 모델링하고, 인간 개발자와 AI 코딩 에이전트가 협력하여 시스템 모델을 스스로 진화시키는 인지 컴파일러(Cognitive Compiler) 방법론을 제안한다.

3.1. 프레임워크 개요 및 과학적 방법론 도입

전통적인 컴파일러가 정적이고 고정된 규칙에 기초해 소스 코드를 기계어로 매핑하는 것과 달리, 본 연구에서 제안하는 인지 컴파일러는 실험 결과를 지식으로 동적 흡수하여 컴파일 제약 규칙을 점진적으로 갱신한다. 이는 GPU 커널 튜닝의 복잡한 설계 공간 속에서 시행착오(Trial-and-Error)를 소거하고 시스템 지식을 체계적으로 누적하기 위한 과학적 방법론 사이클에 기초한다.

과학적 방법론 단계	프레임워크 대응 요소 및 실행 메커니즘
가설 수립 (Hypothesis)	Architect가 시스템 모델과 전역 지식 베이스를 바탕으로 최적화 방향 및 탐색 범위 정의
실험 설계 (Experiment Design)	TechLead가 구체적인 코드 변형 패치 제안 및 컴파일 옵션 명세화
실험 수행 (Execution)	Executor가 타겟 GPU 머신 상에서 배포된 패치를 빌드/실행하고 텔레메트리 데이터 계측
데이터 분석 (Analysis)	TechLead가 수집된 실행 로그 및 Nsight 물리 메트릭을 분석하여 실험 보고서 작성
성찰 및 지식 갱신 (Reflection)	Architect가 인과관계를 종합적으로 평가하고 획득한 한계 지식을 지식 베이스에 영속화(Reflection)

표 3. 커널 엔지니어링의 과학적 방법론 사이클 매핑

3.2. 3계층 위계 구조 및 통제 경계 (Three-Tier Architecture & Boundaries)

본 프레임워크는 정보의 추상화 수준, 실행 권한 및 하드웨어 접근 수준에 따라 Architect(상위), TechLead(중위), Executor(하위)의 3계층 위계로 분할되어 작용한다. 이는 과학실험실 팀의 역할

분담을 모방하여 각 에이전트와 인간의 행동 통제 영역을 엄격하게 규정한다.

■ 3계층 공동 설계 위계 구조 및 통제 경계

1. 상위 계층: Architect (인간 개발자 + 클라우드 LLM)

- 역할: 시스템 궁극 의도(Intent) 정의, 전역 제약 수립, 지식 최종 승인.
- 통제 경계: 타겟 머신의 로컬 코드 실행에 직접 개입하지 않으며, 거시적 구조 설계와 추상화된 물리 데이터 해석에만 집중.

▼ I2E Protocol (Intent-to-Experiment)

2. 중위 계층: TechLead-Agent (Antigravity CLI 코딩 에이전트)

- 역할: 지식베이스 규칙과 가설을 소스코드 패치(``patch_proposals/``) 및 튜닝 탐색 범위로 번역(Translation).
- 통제 경계: 아키텍처 불변성과 컴파일러 제약 조건의 일관성 수호에 전념하며, 타겟 머신에서 코드를 직접 컴파일/실행하지 않음.

▼ S2E Protocol (Spec-to-Execution)

3. 하위 계층: Executor (인간 개발자 + 로컬 실행 환경 + Nsight)

- 역할: 패치를 물리적으로 로드하여 GPU 상에서 JIT 컴파일 및 론칭, 실행 에러에 대한 로컬 자율 디버깅 수행, 프로파일 텔레메트리 계측.
- 통제 경계: 전역적 아키텍처 설계에는 관여하지 않으며, 오직 로컬 오류 극복과 정량 데이터(``run_trace.log``) 수집에 한정.

3.3. 비동기 협업 및 진화 순환 프로토콜 (I2E / S2E / T2K)

각 계층은 명시적인 문서 전달 프로토콜인 I2E(Intent-to-Experiment), S2E(Spec-to-Execution), T2K(Telemetry-to-Knowledge)를 활용해 비동기적으로 상호작용한다. 전체 사이클의 순환과 진화 흐름은 다음과 같다.

- Intent-to-Experiment (I2E):** Architect는 전역 지식 베이스를 참고하여 검증할 가설을 수립하고, 이를 ``.cognitive/sessions/current_task.yaml`` 파일로 직렬화하여 TechLead에게 하달한다.
- Spec-to-Execution (S2E):** TechLead는 지식 규칙을 준수하며 Q, K, V 텐서 로직을 재작성하여 코드 패치 명세를 생성하고, JIT 컴파일 인자들을 포함한 최적화 사양을 ``patch_proposals/`` 디렉토리에 직렬화해 배포한다.

3. **Telemetry-to-Knowledge (T2K)**: Executor는 패치를 반영하여 타겟 GPU에서 빌드를 기동하고, Nsight Compute 물리 데이터 및 하드웨어 성능 지표를 ``.cognitive/execution_logs/run_trace.log``에 기록한다. TechLead는 이 텔레메트리 인과관계를 분석하여 정량적 성과와 병목 한계 원인을 진단한 ``.experiment_report.md``를 작성한다.
4. **지식 성찰 및 영속화 (Reflection)**: Architect는 보고서를 검증한 후 발견된 물리적 임계 지식을 지식 베이스(``.v7.dsl.md``)의 영구 제약 규칙으로 추가 기재한다.

💡 자기 진화(Self-Evolution) 및 실패 학습 메커니즘

본 프레임워크는 실패한 최적화 시도(예: 자원 초과로 인한 컴파일 실패 또는 스페이링)를 유기적으로 학습한다. 컴파일 실패 시 발생한 오류 로그 및 병목 요인은 T2K 프로토콜을 타고 올라와 ``.v7.dsl.md``의 새로운 **물리 제약 조건**으로 영구 주입된다. 이를 통해 컴파일러와 에이전트는 다음 최적화 세션에서 실패했던 파라미터 조합을 탐색 공간에서 원천 격리함으로써, 시스템 스스로 진화하는 **Evolutionary Closed Loop**를 완성한다.

3.4. 물리 제약 정밀 탐색 실증 사례 (Register Spilling)

본 방법론의 우수성은 FMHA(Fused Multi-Head Attention) 커널 설계 도중 레지스터 스페이링(Register Spilling)을 탐지하여 타일 크기를 동적으로 조율한 과정에서 실증된다. 초기 I2E 설계 단계에서 TechLead는 연산 처리의 대역폭을 극대화하고자 128x128 타일 크기로 이식한 FMHA 코드를 S2E 프로토콜로 배포하였다. 그러나 타겟 GPU 환경에서 Executor가 기동한 Nsight Compute 텔레메트리(T2K) 결과, 타일 크기로 인해 스레드당 가용한 최대 레지스터 한계치(255개)를 초과하여 연산 데이터가 VRAM/L1 캐시 계층으로 대량 누수(Spill)되고 있음이 밝혀졌다.

이를 수신한 TechLead와 Architect는 실패 로그에 담긴 하드웨어 임계 정보를 반영하여 "RTX 5070 환경에서 128 타일은 스페이링을 촉발하여 유해함"을 인지적 제약으로 ``.v7.dsl.md``에 등재하였다. 이후 탐색 차원을 64×64 타일링 구조로 하향 보정한 패치를 즉각 재론칭함으로써 레지스터 사용을 최적화하고 스페이링을 원천 소거하여, 최종적으로 125.89 TFLOPS의 최고 성능 도달을 견인하였다.

3.5. 에이전트 협력 설계의 재현 가능성 및 시스템 명세 (Agent Co-design Reproducibility and System Specifications)

인간 개발자와 AI 코딩 에이전트 간의 공동 설계 방법론이 단일 연구실의 단발성 성공을 넘어 학술적 및 실용적으로 **재현 가능(Reproducible)**하기 위해서는, 구체적으로 운용된 대규모 언어 모델 스택,

프롬프트 주입 설계(Meta-prompting), 그리고 상태를 기록하고 추적한 의사결정 로그(Decision Trace Log) 구조를 엄격하게 기술할 수 있어야 한다.

3.5.1. 계층별 기동 모델 스택 (Foundation Models Stack)

본 최적화 세션에서는 연산 자원의 규모와 요구되는 인지적 추상화의 깊이에 따라 파운데이션 모델 스택을 이원화하여 가동하였다.

- **상위 아키텍트(Architect) 엔진:** 거시적인 추론 엔진 통합 정책과 L2 캐시 상주 한계, 메모리 할당자 특성 등 다차원적 아키텍처 결정을 수행하기 위해 클라우드 연동형 초대형 모델인 **Google Gemini 1.5 Pro**를 기동하였다.
- **중위/하위 런타임 에이전트(TechLead) 엔진:** cuTile 코드 변형, 구문 검증, Nsight Compute 텔레메트리 메트릭 파싱 및 패치 생성 등 고도의 툴 호출(Tool Call) 및 context window 실시간 검색을 고속으로 완수하기 위해 **Google Gemini 1.5 Flash**를 백엔드로 가동하였다.

3.5.2. 지식 주입형 메타 프롬프트 설계 (Knowledge-Guided Prompt Design)

에이전트가 cuTile 커널 코드를 재작성하거나 JIT 컴파일 병목을 회피할 때, 컴파일러 제약 조건과 하드웨어 정보를 안정적으로 인지시키기 위해 설계된 컨텍스트 메타-프롬프트(Meta-prompt) 템플릿의 규격은 다음과 같다.

```
# TechLead 에이전트에 인가되는 지식 컨텍스트 주입 메타 프롬프트
```

```
SYSTEM_PROMPT = """
```

```
You are acting as a TechLead Agent within a 3-tier kernel co-design framework.  
Your role is to translate optimization intents into concrete cuTile code patches.
```

```
[Hardware Context: NVIDIA GeForce RTX 5070]
```

- SM Count: 48 SMs
- Register File Size: 255 registers per thread max
- L2 Cache Size: 48.0 MB
- Shared Memory: Max 99 KB per Thread Block

```
[Active System Knowledge & Constraints]
```

- Rule 1 (Memory Allocation): NEVER introduce dynamic padding to avoid PyTorch Cache
- Rule 2 (Register Limit): Matrix tiling block dimensions must satisfy: $TILE_M * TILE_N \leq 255$
- Rule 3 (Prefill Saturation): Batch-1 prefill occupancy saturates at 32 SMs. Limit

```
[Task Intent]
```

```
Rewrite the target attention kernel to optimize dynamic decode ITL under the active  
"""
```


이 메타 프롬프트는 에이전트가 로컬 실행 도중에 발생할 수 있는 시행착오 범위를 사전에 물리/시스템 규칙을 통해 제한함으로써, 탐색 공간을 극적으로 축소하고 Windows Native 최적화 코드로 단일 수렴할 수 있도록 유도한다.

3.5.3. 의사결정 로그 직렬화 규격 (Decision Trace Logging)

실험의 추적성과 재현성을 물리적으로 입증하기 위해, Architect와 TechLead 간의 의사결정 경로는 `.cognitive/sessions/current\_task.yaml` 파일로 매 이터레이션마다 정형화되어 직렬화 및 보존된다.

```
# .cognitive/sessions/current_task.yaml 실제 기록 사양 예시
session_id: "session_003_micro_vllm_tuning"
target_kernel: "nanovllm/layers/attention.py"
hypotheses:
  - "Green Contexts SM partitioning (32:16) prevents L2 cache eviction of weights during tuning"
tuning_parameters:
  prefill_sm: 32
  decode_sm: 16
  tiling_size: [64, 64]
telemetry_requirements:
  - "Decode P50 ITL (ms)"
  - "Total E2E Throughput (tok/s)"
acceptance_criteria:
  throughput: "> 450.0 tok/s"
  p50_itl_reduction: ">= 10.0%"
decision_trace:
  - step: 1
    action: "Launch Eager serving vs Padded serving comparison"
    outcome: "Padded serving resulted in 67% throughput drop due to allocator thrash"
    reflection: "Reject dynamic padding; commit Rule 1 to knowledge base."
```

이 구조적 로그는 개발 세션이 리셋되거나 다른 GPU 하드웨어 환경으로 전이될 때, 에이전트가 과거의 실패 사례와 결정 가설을 즉각 조회(Instant lookup)하여 동일 오류를 반복하지 않도록 방어하는 강력한 상태 보존 수단으로 기능한다.

4. micro-vllm 아키텍처 및 핵심 최적화 (micro-vllm Architecture & Optimizations)

본 장에서는 Windows OS와 단일 소비자용 GPU 환경에 맞추어 마이그레이션하고 최적화된 micro-vllm 서빙 엔진의 핵심 구조적 개선 사항을 기술한다.

4.1. Gloo 기반 분산 통신 백엔드 이식

vLLM과 같은 프로덕션 서빙 프레임워크는 대규모 모델의 분산 연산을 위해 Linux 계열에서만 작동하는 NCCL 백엔드를 기본적으로 사용한다. Windows 네이티브 환경에서 분산 텐서 연산의 컴파일 호환성을 보장하기 위해, 우리는 PyTorch Distributed of **Gloo 백엔드**를 다중 디바이스 분산 스택에 이식하였다.

```
import os
import torch.distributed as dist

# Windows 환경이거나 단일 GPU 연산 시 Gloo 백엔드로 자동 폴백
backend = "gloo" if (os.name == "nt" or world_size == 1) else "nccl"
dist.init_process_group(backend, init_method="tcp://localhost:2333", world_size=world_size)
```

이를 통해 분산 통신 계층의 대대적인 코드 수정이나 이진 컴파일(Binary compilation) 충돌 없이, 텐서 병렬 선형 레이어(Linear Layer)의 논리 구조를 Windows 네이티브 실행 환경으로 완전히 투명하게 마이그레이션할 수 있었다.

4.2. 벡터화된 논리-물리 블록 캐시 재구성

실제 서비스 루프 내에서 prefix-cached prefill 연산(기존에 보존된 역대 프롬프트 이력 캐시와 신규 사용자 입력을 융합하는 단계)을 수행할 때, 새롭게 인가된 Key, Value 텐서 모양(Shape)과 메모리 풀 내부의 논리적 시퀀스 크기 간의 불일치로 인한 런타임 셰이프 오류(Runtime shape mismatch)가 빈번히 유발된다.

우리는 이 셰이프 매칭을 CPU 연산 루프를 사용하지 않고 GPU 상에서 병렬적으로 처리하기 위해, **벡터화된 논리-물리 캐시 재구성 레이어**를 제안하고 적용하였다. context 길이 *seqlen_k*에 해당하는 메모리 오프셋 주소를 텐서 인덱스 연산으로 변환하여 물리 주소로 고속 매핑한다.

```
# GPU 상에서 dynamic logical index를 physical offset으로 고속 매핑
idx_seq = torch.arange(seqlen_k, device=block_table.device)
logical_blk = torch.div(idx_seq, block_size, rounding_mode='floor')
offset = idx_seq % block_size
physical_blk = block_table[b, logical_blk].long()
```

```
k_4d[b, :, :seqlen_k, :] = k_cache[physical_blk, offset].transpose(0, 1)
v_4d[b, :, :seqlen_k, :] = v_cache[physical_blk, offset].transpose(0, 1)
```

이 벡터화된 배치 재배치 로직은 GPU 오버헤드를 sub-millisecond 단위로 격하시켜 호스트 병목을 차단하고, dynamic continuous batching 상황에서의 형상 무결성을 완벽히 보장한다.

4.3. 단일 커널 최적화 (Tiled GEMM & FMHA 64x64)

FMHA(Fused Multi-Head Attention) 커널 설계 시, 메모리 계층 간의 IO 비용을 축소하기 위해 중간 연산 데이터인 attention score를 공유 메모리 및 레지스터 영역에 상주시켰다. online softmax 알고리즘을 타일 레벨에 주입하여 수치 안정성을 취하고, 디코딩과 prefill 단계의 효율성을 도모하는 causal masking을 cuTile 스프레드-블록 연산에 통합하였다.

5. 호스트-디바이스 인터페이스 최적화 및 자원 격리 (Host-Device & Resource Isolation)

5.1. JIT 컴파일 스톰과 메모리 할당자 스래싱 딜레마 (JIT Compilation Storm vs. Allocator Thrashing)

JIT 컴파일 기반 Python GPU DSL인 cuTile은 입력으로 전달되는 텐서 형상의 기하학적 변화에 반응하여 실시간으로 PTX 바이너리를 즉시 재빌드한다. continuous batching 서빙 환경 하에서는 동적으로 매 디코드 스텝마다 배치 크기와 시퀀스 길이가 변화하므로 JIT 컴파일이 무수히 유발되는 *JIT Compilation Storm*이 심각한 성능 지연을 만든다.

이러한 JIT 컴파일의 발생 빈도를 억제하기 위해, 우리는 텐서 형상의 차원을 256의 배수로 강제 정렬하는 **dynamic padding** 기법을 튜닝 옵션으로 적용해 보았다. 그러나 벤치마크 결과는 예상을 완전히 벗어나 성능이 급격히 저하되었다.

이 현상의 미시적 원인을 규명하기 위해 PyTorch의 **CUDA 캐싱 할당자(Caching Allocator)**의 블록 재사용(Block Reusability) 메커니즘을 정밀 추적하였다. cuTile/Triton 등 JIT 컴파일 엔진은 컴파일이 완료된 커널을 내부 해시 테이블(Hash Table)에 적재하여 캐싱하므로, 동일한 Shape이 유입될 때 컴파일 없이 수 μ s(마이크로초) 내에 끝나는 해시 룩업(Lookup) 지연만을 유발한다. 반면, dynamic padding을 우회 통과시키기 위해 매 디코드 스텝마다 가변 텐서를 256의 배수로 정렬하여 대형 패딩 텐서를 매번 새로 할당(New Allocation)할 경우, PyTorch의 캐싱 할당자는 내부 이중 연결 리스트(Doubly Linked List) 스캔을 반복 수행하여 호스트 CPU 연산 병목을 유발한다.

더 큰 문제는 캐시 히트에 실패(Cache Miss)할 때 발생한다. 크기가 정확히 일치하는 캐시 블록이 없을 경우, OS 디바이스 드라이버 커널로의 다이렉트 드라이버 콜(Driver Call)인 `cudaMalloc` 이 동적으로 강제 호출되어 수십 μ s의 지연을 일으키고, 연이어 할당된 VRAM 물리 영역을 0으로 덮어쓰는 `cudaMemset` 초기화 연산으로 인해 디바이스 내부에 추가적인 동기화 대기가 촉발된다. 또한 Python의 Reference Count 감축에 따른 가비지 컬렉터(GC) sweep이 돌 때마다, 미사용 캐시 해제 및 메모리 동기화를 위해 무거운 `cudaDeviceSynchronize` 가 강제로 수반되어 ms 단위의 런타임 스파이크가 루프에 누적된다.

나아가, Eager-mode LLM Serving 하에서의 **암묵적 동기화(Implicit Synchronization)** 병목이 추가로 규명되었다. dynamic padding을 가하면 실제 활성 배치(예: 배치 크기 100)보다 더 큰 영역(예: 256의 배수인 256)으로 텐서가 확장되며 156개의 더미(Dummy) 토큰 텐서 영역이 강제 생성된다. 이 더미 영역을 마스킹하고 인덱싱하여 `torch.cat` 및 결합 연산을 수행하는 과정에서 Host(CPU)와 Device(GPU) 간의 즉각적인 메모리 복사 및 참조 검증이 강제되며, 이로 인해 PyTorch가 기본 CUDA 스트림(Default Stream)을 차단/락킹(Locking)하게 되어 실제 GPU 상의 연산 스케줄링을 연쇄적으로 방해하는 간섭(Interference) 부작용이 실증되었다.

vLLM이나 TensorRT-LLM과 같은 프로덕션 추론 프레임워크는 CUDA Graphs를 사용해 메모리 풀 할당 자체를 실행 루프 바깥 영역으로 사전 캡처(Capture)하여 이를 은폐한다. 하지만 CUDA Graph 캡처가 차단된 Windows 네이티브 **Eager-mode serving** 환경에서는 이 메모리 관리 부하가 시스템 전체 성능을 가로막는 절대적인 병목으로 표출된다. 즉, *"dynamic batching 환경에서 JIT 오버헤드를 완화하기 위해 적용하는 통상적 기법인 Shape Padding이, Graph가 없는 Windows 네이티브 Eager Serving에서는 오히려 호스트 단의 메모리 스래싱(Malloc + Zero-fill + GC + 스트림 락)을 촉발하여 Throughput을 67.0% 붕괴시키는 파괴적인 독(Poison)이 된다"*는 시스템적 반직관을 실증으로 드러내었다.

5.2. CUDA Green Context 기반 SM 자원 분할 및 캐시 지역성 최적화

동적 동시 서빙 워크로드 하에서는 대량의 토큰을 처리하는 무거운 Prefill 연산이 개시될 시 GPU의 컴퓨팅 파워를 독점하여, 지연시간에 고도로 민감한 일반 클라이언트들의 디코드 연산이 중단되거나 지연이 심해지는 리소스 간섭 현상이 초래된다.

이를 예방하기 위해, 우리는 CUDA Python 1.0의 최신 하드웨어 분할 API인 **Green Contexts**[5]를 적용하여 RTX 5070 GPU의 전체 48개 SM 자원을 물리적으로 강제 파티셔닝하였다.

- **Prefill Context (32 SMs):** 막대한 부동소수점 연산 처리가 동반되는 프롬프트 분석용 SM 그룹.
- **Decode Context (16 SMs):** 실시간 응답 지연을 방어하기 위한 1토큰씩의 생성용 전용 SM 그룹.

```
from cuda.core import Device, ContextOptions, SMResourceOptions

# RTX 5070 GPU의 48개 SM을 각각 32개, 16개로 물리 분리
dev = Device(0)
sm = dev.resources.sm
long_grp, crit_grp = sm.split(SMResourceOptions(count=(32, 16)))[0]
ctx_prefill = dev.create_context(ContextOptions(resources=[long_grp]))
ctx_decode = dev.create_context(ContextOptions(resources=[crit_grp]))
```

이러한 물리적 SM 구획화는 GPU 하드웨어 레벨에서 **L2 캐시 스레시(L2 Cache Thrashing)** 및 **데이터 방출(Eviction)**을 억제하는 미시적 메커니즘으로 작동한다. RTX 5070 GPU는 48.0 MB의 통합 L2 캐시를 모든 SM이 공유한다. 대용량 텐서 연산을 수행하는 Prefill 단계는 넓은 메모리 풋프린트를 탐색하기 때문에 막대한 양의 L2 캐시 라인을 요구하며, 기존에 적재되어 있던 디코드 스텝의 모델 가중치(Weights) 및 KV 캐시 블록 데이터들을 L2 캐시 공간 밖으로 급격히 Evict 시킨다.

이후 시작되는 디코드 커널은 메모리 대역폭 제한(Memory-bandwidth bound) 특성을 가지므로 L2 캐시 상에 가중치가 유지(Residency)되어야 연산 지연이 최소화되는데, Prefill에 의한 eviction 부작용으로 인해 매번 VRAM 글로벌 메모리에서 데이터를 다시 로드하는 심각한 병목을 겪게 된다.

Green Contexts를 통해 디코드 연산에 할당된 SM을 16개로 강제 격리하면, Prefill 커널이 연동되는 동안 Decode SM 영역의 하드웨어 L2 캐시 बैं크 점유권(Cache Bank Allocations)과 메모리 큐잉 리소스를 분리 보호하는 효과를 얻게 된다. 즉, Prefill의 무거운 쓰기 스트림이 Decode 가 상주시키는 가중치 데이터를 침범해 지워버리는 간섭(Interference)을 차단하여, 디코드 연산 시 데이터 로드가 VRAM 버스로 우회하지 않고 L2 캐시 히트(L2 Cache Hit)로 다이렉트 처리되는 미시적 캐시 로컬리티 향상을 완성한다.

6. 실험 및 평가 (Evaluation)

6.1. 실험 환경 및 시나리오

본 논문에서 제안한 시스템의 실증을 위해 Windows 11 OS 환경에서 NVIDIA GeForce RTX 5070 GPU(48 SM, 12GB VRAM, CUDA 13.3) 및 Qwen2.5-3B-Instruct 모델을 사용해 평가를 실시하였다. 동시 사용자 수는 256명으로 고정하고, 입력 및 출력의 크기를 100에서 1024 토큰 범위 내에서 동적으로 조율하는 continuous batching 상황을 재현하였다. 최악의 자원 경합 상황을 시뮬레이션하기 위해, 100토큰 디코드 클라이언트가 생성 연산을 정상 구동하는 중 5번째 스텝에서 2048토큰의 대규모 prefill 요청을 강제 투입하여 지연 시간 및 처리량을 모니터링하였다.

6.2. 패딩 여부에 따른 할당 오버헤드 실증 (Eager vs Padded)

표 1은 JIT 컴파일 방지를 위한 dynamic padding 기법(Padded cuTile)과 본 연구가 제안하는 무 패딩 Eager 할당 기법(Eager cuTile) 간의 E2E 총 처리량(Throughput) 비교 결과이다.

시스템 아키텍처 설정	처리 총 토큰 수 (개)	총 소요 시간 (초)	생성 처리량 (tok/s)	성능 대조 비율 (%)
Eager cuTile (No Padding - 본 연구)	133,966	284.54	470.82	기준치 (100.0%)
Padded cuTile (dynamic padding)	133,966	863.39	155.16	-67.0% (성능 하락)

표 1. 세이프 패딩 적용 유무에 따른 E2E 서빙 효율성 비교

실험 측정 결과, dynamic padding 설정은 Eager 설정에 비해 전체 시스템 Throughput이 67.0% 하락하는 최악의 병목을 유발하였다. 이는 매 스텝 변동하는 배치 크기에 부응하기 위해 대형 패딩 텐서를 호스트 CPU 단에서 계속 신규 생성하고 복사하는 오버헤드가 cuTile JIT 컴파일 록업 오버헤드에 비해 시스템 전체 성능에 치명적인 영향을 끼침을 직접적으로 반증한다.

6.3. Green Contexts 적용에 따른 성능 비교

표 2는 디코드 전용 SM 자원을 고정 분할한 Target(Green ON) 설정과 기존 Baseline(Green OFF) 설정 간의 4대 주요 서빙 품질 성능 비교 결과이다.

평가 성능 지표 (Metric)	Baseline (Green OFF)	Target (Green ON)	개선 비율 (%)
TTFT (Prefill Latency)	244.60 ms	243.42 ms	-0.5% (유지)
Decode P50 ITL (Median)	48.63 ms	41.83 ms	-14.0% (단축)
Decode P99 ITL (Tail)	85.31 ms	84.34 ms	-1.1% (안정화)

평가 성능 지표 (Metric)	Baseline (Green OFF)	Target (Green ON)	개선 비율 (%)
Total Throughput (총 처리량)	415.50 tok/s	459.73 tok/s	+10.6% (향상)

표 2. Qwen2.5-3B-Instruct 기반 Baseline vs Green Contexts 성능 비교

측정 결과, Green Contexts를 활성화하여 SM 자원을 Prefill 32개, Decode 16개로 분할 구획한 타겟 설정은 베이스라인 대비 **중간값 Decode 지연시간(P50 ITL)을 14.0% 단축**시키는 성과를 보였다. 더불어 전체 토큰 처리량(Throughput) 역시 **10.6% 증가**하는 고무적인 성능 우위를 보였다. 이는 디코드용 SM 수량을 16개로 강제 한정하여 디코드 어텐션 연산 시 필요한 가중치와 토큰 정보가 L2 캐시 영역에 높은 비율로 residency를 확보할 수 있도록 캐시 스래싱을 차단한 덕분이다.

동시에, Prefill 연산의 가용한 하드웨어 파티션이 48개에서 32개로 감소했음에도 불구하고 TTFT(First Token Latency)의 지연 시간 증가는 단 -0.5% 수준으로 완벽하게 제어되었다.

이 현상의 원인은 ****Persistent CTA (Cooperative Thread Array) 스케줄링의 포화점(Saturation Point)****에 있다. Batch 크기가 1인 Prefill 워크로드는 아무리 시퀀스 길이가 2048 토큰일지라도 하드웨어 구조상 론칭되는 스레드 블록의 총개수(Grid Size)가 제한적이다. 따라서 RTX 5070 GPU가 제공하는 48개의 SM 연산 자원을 전부 소모하여 Occupancy를 채우지 못하며, 약 32개의 SM 자원 수준에서 이미 최대 부동소수점 처리 한계량인 하드웨어 Occupancy Saturation Sweet-spot에 이르게 된다. 결과적으로 남는 16개의 SM을 Decode에 구획 분할하더라도 Prefill Latency(TTFT)의 지연 손실이 거의 유발되지 않으면서도 Decode의 ITL 단축과 캐시 레지던시를 극대화할 수 있었던 정량적 근거를 설명해 준다.

💡 시스템 아키텍처적 시사점 (Systems Insight)

본 실험을 통해 단일 GPU 내에서의 물리적 SM 자원 분할은 단순 성능 개선을 넘어, 이기종 동적 워크로드가 동시 구동되는 추론 서빙 파이프라인에 필수적인 L2 캐시 지역성을 지키는 수단임이 검증되었다. 또한, 본 연구의 개발 전 주기에 걸쳐 코딩 에이전트는 미시적 레지스터 누수 감지 및 물리 캐시 벡터 재구성에 필요한 구체적 텐서 정렬 스크립트를 능동적으로 도출하는 등, 인간 엔지니어의 상위 설계 의도를 실질적인 시스템 프로그래밍 최적화 결과물로 변환하는 데 중추적인 코디자인(Co-design) 가치를 실증해 주었다.

7. 결론 및 향후 연구 (Conclusion & Future Work)

본 논문은 Windows OS 환경에서 cuTile DSL과 CUDA Python 1.0을 결합한 독창적인 고성능 LLM 추론 엔진인 micro-vllm을 구현하고 E2E 수준의 병목 요인을 실증 규명하였다. 특히 코딩 에이전트와의 밀접한 협업 설계 및 미시 프로파일링 피드백을 통해 64x64 타일링과 벡터화된 논리-물리 주소 변환 레이어를 구축하였으며, eager 모드 서빙 시 JIT 방지용 패딩 기법이 가혹한 메모리 할당자 스래싱을 유발하여 Throughput을 67.0% 저하시키는 역효과가 있음을 정량적 벤치마크를 통해 최초 입증하였다. 아울러 Green Contexts 기반 SM 파티셔닝 튜닝을 통해 Decode P50 ITL을 14.0% 단축하는 시스템 개선을 실현하였다.

현재 단일 스레드 순차 엔진 구조 하에서는 시간 축 상에서의 Prefill과 Decode 연산이 완벽하게 독립 병렬 수행되지 못하는 하드웨어 제약이 존재한다. 향후 연구로는 엔진 루프를 비동기 멀티스레드 (Asynchronous Multi-threaded) 아키텍처로 전면 이원화하여, 서로 다른 GPU 스트림과 물리 SM 파티션 상에서 Prefill과 Decode가 오버랩(Overlap)되어 동작하는 완전한 E2E 자원 격리를 구현해 나갈 계획이다.

참고문헌 (References)

1. Kwon, W., Li, Z., Zhuang, S., et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
2. NVIDIA Corporation. "NVIDIA CUDA 13.3 Enhances GPU Development with Tile Programming in C++ Compiler, Autotuning, and Python Updates." *NVIDIA Developer Blog*, 2026.
3. Tillet, P., Kung, H. T., & Cox, D. "Triton: an intermediate language and compiler for tiled neural network computations." *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.
4. Dao, T., Fu, D. Y., Ermon, S., et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
5. PyTorch Foundation. "torch.cuda.green_contexts: Granular Resource Partitioning for CUDA Kernels." *PyTorch Beta Documentation*, 2025.
6. NVIDIA Corporation. "cuTile Python API Reference Guide." *NVIDIA CUDA Documentation*, 2026.

부록 A: cuTile 프로그래밍 모델 핵심 개념 (cuTile Programming Model)

NVIDIA가 도입한 cuTile DSL은 전통적인 CUDA C++의 극단적인 저레벨 복잡성과 PyTorch의 고수준 추상화 사이의 간극을 메우는 타일 기반 프로그래밍 패러다임이다. 주요 동작 아키텍처는 다음과 같은 핵심 개념들을 내포한다.

1. 타일 추상화 (Tile Representation)

cuTile 환경에서 모든 행렬 및 벡터 데이터는 고정된 크기의 기하학적 블록인 **타일(Tile)**로 정의된다. 개발자는 개별 스레드 레벨에서 글로벌 메모리 주소를 하드코딩해 스레드 ID 인덱싱을 연산할 필요 없이, 스레드 블록 단위로 연계된 논리 타일 좌표계를 제어한다. JIT 컴파일러는 이 타일 구성을 수신하여 각 GPU의 레지스터 구조에 맞춰 루프 전개(Loop unrolling), 벡터 로드/스토어 지시어 생성, 스레드 레이아웃 배치를 백그라운드에서 전 자동화한다.

2. MMA (Matrix Multiply-Accumulate) 하드웨어 제어

cuTile은 Tensor Core 하드웨어가 직접 처리하는 고속 행렬 연산 지시어인 MMA를 ``ct.mma``로 래핑하여 제공한다. 이는 레지스터 타일 내에서 독립적으로 수행되는 $D = A \times B + C$ 누적 연산을 의미한다. 글로벌 VRAM 메모리 버스를 거치지 않고 순수 레지스터 파일(Register File) 상에서 전량 연산이 완결되므로 연산 능력이 GPU 메모리 대역폭 한계와 별개로 최대로 가동된다.

3. 협력적 메모리 로드 및 스토어 (Cooperative Memory Operations)

VRAM 글로벌 메모리와 스레드 레지스터 타일 간의 데이터 복사는 협력적 로드(``ct.load``) 및 스토어(``ct.store``) 명령어를 통해 블록 내 스레드들이 공유 메모리를 매개로 공동 집행한다. 특히 cuTile은 sequence length가 타일의 기본 블록 차원(예: 64)과 완전 정렬되지 않고 어중간하게 남은 자투리 영역($T < 64$)에 대해 ``ct.PaddingMode.ZERO``와 같은 제로 필링 패딩 모드를 네이티브 제공하여, CUDA 개발 시 흔히 마주하는 Out-of-bounds 메모리 접근 결함을 안전하게 원천 차단해 준다.

4. Compute Capability 9.0 (Hopper) 아키텍처 이식성

CUDA 13.3에 탑재된 최신 컴파일러 엔진은 Compute Capability 9.0 이상에서 도입된 Hopper 아키텍처 전용 텐서 코어 가속 및 비동기 트랜잭션 메모리 제어 규칙을 cuTile DSL로 완벽 호환한다. 이로써 단일 소비자용 RTX GPU에서 컴파일되는 동일한 Python 코드가 엔터프라이즈 전용인 H100/H200 머신으로 진입 시, 하드웨어 명령어의 재생성이나 아키텍처 포팅 공정 없이 Hopper 네이티브 고성능 파이프라인으로 100% 투명하게 컴파일을 승계할 수 있게 된다.

부록 B: LLM 추론 동작 원리 핵심 개념 (LLM Inference Concepts)

LLM 서비스 파이프라인을 효율적으로 최적화하기 위해서는 생성 추론 모델의 미시적/거시적 연산 특성을 정확히 이해하는 것이 선행되어야 한다.

1. Prefill 단계와 Decode 단계의 극적인 하드웨어 대조

- **Prefill Phase (프롬프트 연산)**: 사용자가 주입한 입력 텍스트 전체를 동시에 파싱하는 단계이다. Query, Key, Value 간의 대규모 행렬 곱셈이 융합 수행되므로, 대량의 병렬 플롭스(FLOPs) 연산량이 필요한 **연산 성능 제한적(Compute-bound)** 성격을 가진다.
- **Decode Phase (토큰 생성)**: 이전까지 생성된 모든 토큰 정보를 기반으로 오직 1개의 다음 토큰을 순차 예측하는 단계이다. 매번 Query 벡터(길이 1)와 대규모 KV 캐시 행렬 간의 벡터-행렬 곱셈(GEMV)이 이루어지며, 모델의 방대한 가중치 파라미터를 VRAM 메모리에서 지속적으로 다시 읽어 들여야 하므로 ****메모리 대역폭 제한적(Memory-bandwidth bound)**** 특성이 시스템 병목을 지배한다.

2. KV 캐싱 (KV-Caching) 및 Paged KV Cache

자기회귀 예측에서 과거 토큰들의 Key, Value를 매번 처음부터 다시 연산하게 되면 스텝이 반복될수록 계산 복잡도가 시퀀스 길이에 비례해 제곱배($O(N^2)$)로 누적 증가하는 치명적인 성능 저하가 일어난다. 이를 차단하고자 과거 토큰의 연산 결과인 Key, Value 벡터들을 VRAM 상에 정적 보존해 두는 것이 **KV 캐시**이다.

그러나 dynamic serving 환경에서 가변 시퀀스의 크기에 맞춰 가상 메모리를 통째로 사전 할당하면 극심한 외부/내부 메모리 파편화(Fragmentation)가 발생한다. **Paged KV Cache**는 OS의 가상 메모리 기법을 빌려 VRAM의 캐시 영역을 일정한 블록(Block) 단위로 물리 페이지로 분할하고, 논리 주소를 block table을 거쳐 매핑 관리함으로써 VRAM 단편화를 완전히 차단하는 vLLM의 핵심 기반 기술이다.

3. Continuous Batching (연속 배치 처리)

기존의 정적 배치(Static Batching)은 배치에 속한 모든 사용자 시퀀스들의 생성이 전량 종결될 때까지 대기해야 하므로 자원의 극심한 낭비를 초래했다. **Continuous Batching**은 각 시퀀스가 개별적으로 종료되면 즉시 새로운 요청을 동적으로 교체 투입하여, 배치 단위가 아닌 토큰 스텝 단위의 유연성을 제공하는 기법이다.